안드로이드 프로젝트 #1

공통 주제 : SNS

김갑석 2026-03-12

#프로젝트 진행 일정

1주차 (Design) 통합 설계 및 환경 구축

기획 및 분석 , UI/UX 전체 설계 ,전체 기능 설계 , 아키텍처 및 환경 셋업

2주차(Core) : MVP 핵심 기능 구현

핵심 도메인 개발 ,비즈니스 로직 연동 ,GitHub 기반 협업 ,중간 점검

3주차(Enhance) 부가 기능 및 완성도 향상

부가 기능 구현 ,UI/UX 고도화 ,리팩토링

4주차(Final) : 최종 검수 및 성과 공유

QA 및 배포 ,최종 문서화 ,최종 발표 및 회고

Overview

1. 팀 구성
2. 기획
3. 설계
4. 개발 환경 구축
5. 기능 개발
6. 프로젝트 검증 및 리팩토링
7. 배포 준비 및 최종 마무리
8. 최종 발표

1. 팀 구성

- 팀 운영 원칙

  모든 팀원은 기획 부터 발표까지 전 과정에 함께 참여합니다.

  역할을 나누는 것에 그치지 않고, 각 분야의 결과물 품질에 대해 전담 책임자(Master)를 지정하여 협업합니다.

- 스크럼 마스터 선정 (팀장)

  프로젝트 리듬 및 품질 책임

  - 스프린트 일정 관리
  - 회의 진행 및 진행 상황 점검
  - GitHub 이슈 / PR 흐름 관리
  - 팀 커뮤니케이션과 일정 조율

1. 팀 구성 - 책임 분담 예시

- 기획 및 화면 설계 책임

  서비스 기능 정의, User Flow 설계, **기능 정의서 실시간 최신화**

- UI/UX 개발 책임

  주요 UI 위젯/레이아웃 구현 주도, 디자인 시스템 및 **Figma 관리**

- 기능 개발 책임

  비즈니스 로직(상태 관리), API 및 Firebase 데이터 연동 관리

- 코드 구조 및 품질 관리 책임

  폴더 구조 설계, GitHub 브랜치/PR 전략 수립, **코드 리뷰 주도**

- 문서 및 발표 준비 책임

  회의록/README 관리, **트러블슈팅 기록**, 발표 자료 및 리허설

2. 기획

- 프로젝트 기획
  - 공통 대주제 : SNS 앱
  - 팀별 소주제 : 팀만의 특화된 색깔을 갖는 주제를 한줄로 정의 하기
    - 예) “초보 캠퍼들을 위한 실시간 노하우 공유 SNS”
- 프로젝트 주제 예시
  - 사진으로 감정을 공유하는 SNS
  - 캠핑 노하우 커뮤니티
  - 런닝 기록공유 커뮤니티/SNS
  - 반려동물 커뮤니티/SNS

2. 기획 - 문서화

- 핵심 기능 정의 및 우선순위 설정

  함께 앱에 필요한 기능을 나열하고, 우선순위를 결정합니다.

- 기능 예시
  - 핵심 기능
    - 회원가입 / 로그인
    - 피드(게시글) 목록
    - 글 작성 / 댓글
  - 부가 기능
    - 좋아요, 알림등

  프로젝트 기간과 리소스를 고려하여 핵심 기능(필수)과 부가 기능(선택)으로 구분합니다.

- 산출물 : 프로젝트 기획안(서비스 정의, 타겟, 기능 리스트)

2. 기획 - 프로젝트 기획안 예시

프로젝트 기획안

1.서비스 개요

서비스명 : XXX

한 줄 정의 : 초보 캠퍼들을 위한 실시간 노하우 공유 및 장비 리뷰 커뮤니티

서비스 배경 : 캠핑 입문자가 늘어나고 있지만, 현장 상화(날시, 장비 사용법)에 대한 실시간 정보를 얻기 어려움.

주요 타겟 : 캠핑을 막 시작한 2030 세대 및 장비 정보 공유를 즐기는 캠핑 마니아

2.핵심 기능 리스트

인증(Auth) : 이메일 및 소셜 로그인, 회원 가입

커뮤니티(Feed) : 캠핑 후기 및 노하우 포스팅(사진, 글), 댓글, 좋아요

통합검색 : 해시태그 기반의 캠핑장 및 장비 리뷰 검색

프로필(Profile): 내 활동 내역(작성 글) 확인 및 프로필 정보 수정

3.부가 기능 리스트

게스글에 댓글, 좋아요 기능

알림(Notification): 내 글에 댓글이 달리거나 좋아요 발생 시 푸시 알림.

팔로우/팔로잉

3. 설계

- 기능 설계 : 기능별 상세 로직 및 예외 상황 정의
- UI/UX 설계 : 사용자 이동 경로(Flow) 및 화면 뼈대(Wireframe) 배치
- 데이터 설계 : Firestore 컬렉션 구조 및 필드 타입 정의
- API 설계 : 외부 데이터 연동 규격 및 Firebase 통신 규칙 정의
- 아키텍처 선정 : MVVM + Featured-based
- 산출물 : 기능 정의서(Notion), User Flow , 와이어프레임 , ERD, API 명세서

3. 설계 - 기능

- 기능 설계의 목적

화면에서 정의된 경험을 실제로 구현할 수 있도록 구체화하는 단계

  - 어떤 기능이 어떻게 작동할 지 정의
  - 데이터 구조 설계
  - 앱의 폴더 구조 및 아키텍처 정리

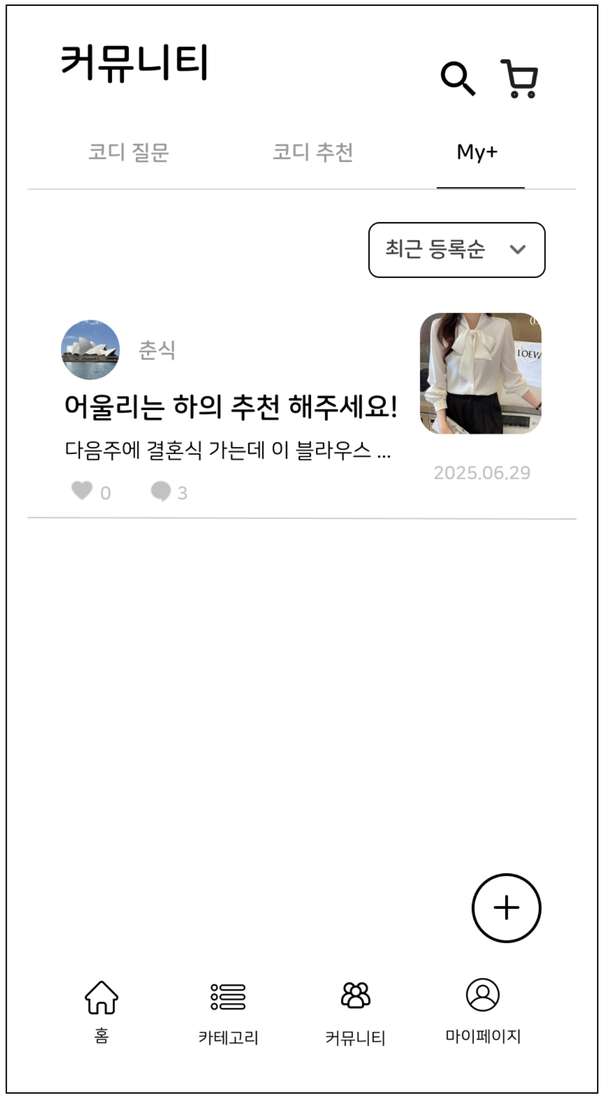
3. 설계 - 기능 정의서 예시

- 기능명 : 사용자 로그인
- 기능 설명 : 사용자가 이메일과 비밀번호로 로그인을 수행한다. 유효한 정보일 경우 토큰을 발급받고 앱 내에서 인증 상태를 유지한다.
- 사용자 : 일반 사용자
- 관련 화면 : 로그인 화면(/login)
- 진입 조건 : 사용자가 앱 실행 후 로그인 버튼 클릭
- 처리 흐름 : 사용자가 이메일, 비밀번호 입력 -> “로그인” 버튼 클릭 -> API 호출:POST /api/auth/login 응답 성공시 accessToken 저장 및 홈화면으로 이동
- 입력 항목 : 이메일, 비밀번호
- 예외 처리 :
  - 이메일 형식이 유효하지 않은 경우 -> 에러 메세지 출력
  - 비밀번호가 틀린경우 -> 에러 메세지 출력
- 성공 조건 : code = “0000” , accessToken, refreshToken 정상 발급되고 홈 화면으로 이동함.
- API 명세 연동 : Swagger : /api/docs/auth/login 참조
- 비고
  - 로그인 유지 기능 포함(DataStore 사용)
  - 로그인 실패 회수 제한(5회 초과시5분 제한)

3. 설계 - UI / UX

- 사용자 흐름 설계(User Flow)
  - 앱의 시작 부터 끝가지 사용자가 이동하는 경로를 시각화
  - Figma(FigJam), Flowchart등
- 화면 명세서 작성
  - 화면별 UI 구성 요소와 해당 요소의 기능(이벤트)정의
  - 각 화면에 대한 와이어프레임(뼈대)제작
    - 버튼 클릭시 이동 경로, 데이터 로딩시 노출될 UI 명시
    - 디자인 시안 보다는 화면의 기능적 구조
- 산출물 : User Flow / 화면 명세서 (와이어프레임 + 각 화면별 기능 설명)

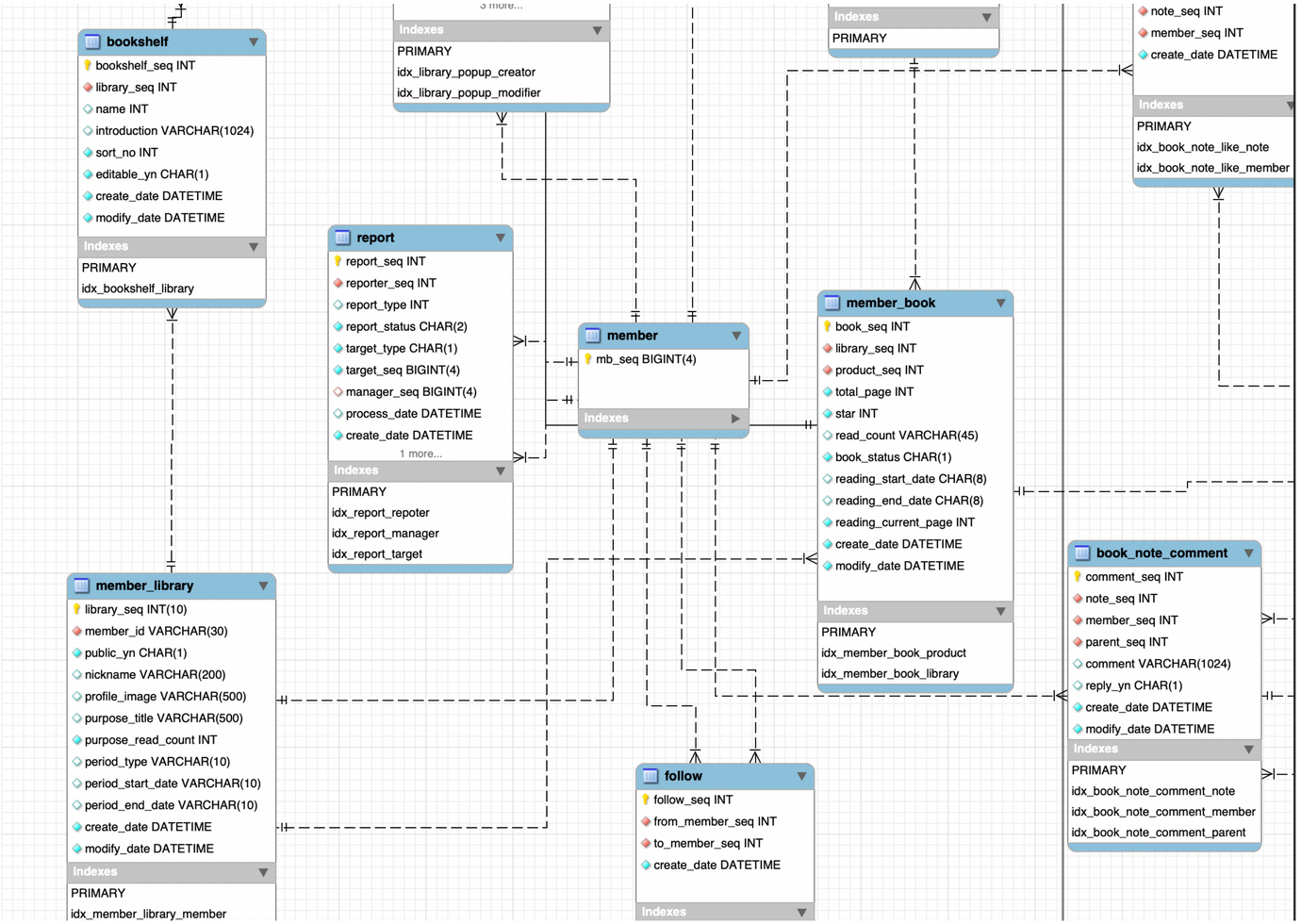
3. 설계 - 화면 상세 명세서 예시



📌 데이터 (posts 컬렉션)
- postId (String)
- userName (String)
- imageUrl (String)
- content (String)
- likeCount (int)
- createdAt (Timestamp)

🎯 액션
- 좋아요 버튼 클릭 → likeCount + 1
- 게시글 클릭 → /feed/detail?postId={postId}
- 프로필 클릭 → /profile?uid={userId}

3. 설계 - ERD 예시


bookshelf
bookshelf_seq INT
library_seq INT
name INT
introduction VARCHAR(1024)
sort_no INT
editable_yn CHAR(1)
create_date DATETIME
modify_date DATETIME
Indexes
PRIMARY
idx_bookshelf_library
Indexes
PRIMARY
idx_library_popup_creator
idx_library_popup_modifier
Indexes
PRIMARY
note_seq INT
member_seq INT
create_date DATETIME
Indexes
PRIMARY
idx_book_note_like_note
idx_book_note_like_member
report
report_seq INT
reporter_seq INT
report_type INT
report_status CHAR(2)
target_type CHAR(1)
target_seq BIGINT(4)
manager_seq BIGINT(4)
process_date DATETIME
create_date DATETIME
1 more...
Indexes
PRIMARY
idx_report_repoter
idx_report_manager
idx_report_target
member
mb_seq BIGINT(4)
Indexes
member_book
book_seq INT
library_seq INT
product_seq INT
total_page INT
star INT
read_count VARCHAR(45)
book_status CHAR(1)
reading_start_date CHAR(8)
reading_end_date CHAR(8)
reading_current_page INT
create_date DATETIME
modify_date DATETIME
Indexes
PRIMARY
idx_member_book_product
idx_member_book_library
member_library
library_seq INT(10)
member_id VARCHAR(30)
public_yn CHAR(1)
nickname VARCHAR(200)
profile_image VARCHAR(500)
purpose_title VARCHAR(500)
purpose_read_count INT
period_type VARCHAR(10)
period_start_date VARCHAR(10)
period_end_date VARCHAR(10)
create_date DATETIME
modify_date DATETIME
Indexes
PRIMARY
idx_member_library_member
follow
follow_seq INT
from_member_seq INT
to_member_seq INT
create_date DATETIME
Indexes
book_note_comment
comment_seq INT
note_seq INT
member_seq INT
parent_seq INT
comment VARCHAR(1024)
reply_yn CHAR(1)
create_date DATETIME
modify_date DATETIME
Indexes
PRIMARY
idx_book_note_comment_note
idx_book_note_comment_member
idx_book_note_comment_parent

3. 설계 - API 명세서 예시 #1

구분	기능명	요청 방식	Endpoint / Path	요청 파라미터	응답(JSON)	설명 / 비고
인증	회원가입	POST	/auth/signup	{ email, password, name }	{ uid, email, createdAt }	Firebase Auth 이용, 중복 이메일 예외처리
인증	로그인	POST	/auth/login	{ email, password }	{ uid, token }	Firebase Auth 로그인 후 JWT 반환
피드	피드 목록 조회	GET	/posts	?limit=10&lastId=	[{ postId, userId, content, imageUrl }]	Firestore 컬렉션 `posts`에서 최신순 조회
피드	피드 작성	POST	/posts	{ content, imageUrl }	{ postId, createdAt }	Storage에 이미지 업로드 후 URL 저장
피드	피드 상세 조회	GET	/posts/{postId}	{ postId }	{ postId, userId, content, comments[] }	Firestore `posts` 단일 문서 조회
댓글	댓글 등록	POST	/posts/{postId}/comments	{ commentText, userId }	{ commentId, createdAt }	Firestore SubCollection `comments` 사용
댓글	댓글 삭제	DELETE	/posts/{postId}/comments/{commentId}	{ commentId }	{ success: true }	본인 댓글만 삭제 가능

3. 설계 - API 명세서 예시 #2(Swagger)

게시판 게시판(QnA, 공지 등) 관련 기능

POST	/posts/	게시글 등록
GET	/posts/	전체 게시글 목록
GET	/posts/users	내가 작성한 게시글 목록
GET	/posts/users/{_id}	지정한 사용자가 작성한 게시글 목록
GET	/posts/{_id}	게시글 상세
PATCH	/posts/{_id}	게시글 수정
DELETE	/posts/{_id}	게시글 삭제
GET	/posts/{_id}/replies	댓글 목록
POST	/posts/{_id}/replies	댓글 등록
PATCH	/posts/{_id}/replies/{reply_id}	댓글 수정
DELETE	/posts/{_id}/replies/{reply_id}	댓글 삭제
GET	/seller/{sellerId}/posts/	판매자의 상품에 등록된 게시글 목록

회원 회원 관리 기능

POST	/users/	회원 가입
GET	/users/	회원 목록 조회
GET	/users/email	이메일 중복 체크
GET	/users/name	이름(name) 중복 체크
POST	/users/signup/oauth	OAuth 인증 후 자동 회원 가입

3. 설계 - MVVM 폴더구조 예시

```
app/
 ├ core/                    # 공통 인프라 (품질 관리 책임)
 |  ├ base/                 # BaseViewModel 등 공통 베이스
 |  ├ di/                   # Hilt/koin 설정
 |  ├ network/              # Firebase/Retrofit 공통 설정
 |  ├ ui/                   # [공통 UI] 디자인 시스템 및 위젯
 |  |  ├ components/        # CustomButton, CustomTextField등
 |  |  └ theme/             # Color, Type, Theme
 |  └ util/                 # 확장 함수 및 유틸리티 함수
 ├ features/
 |  ├ auth/                 # [인증 도메인] 로그인/회원가입
 |  |  ├ data/              # Model, Repository
 |  |  ├ ui/                # Loginscreen, SignupScreen
 |  |  └ viewmodel/         # 상태 관리, 비즈니스 로직(AuthViewModel)
 |  ├ feed/                 # [피드 도메인] 피드(게시글) 관련 기능
 |  |  ├...
 |  |  └ viewmodel/
 |  └ profile/              # [프로필 도메인]사용자 프로필 관련 기능
 |    ├ data/
```

4. 개발 환경 구축

- 개발 환경 설치 및 테스트

  - Android Studio 설치 및 Kotlin 개발 환경 및 테스트

- GitHub 저장소 생성 및 연동

- 프로젝트 초기화

  - MVVM + Feature-based 구조 설계 및 초기 커밋

- 의존성 설정

  - Firebase SDK, Hilt/Koin, Retrofit, Jetpack 관련 설정

- 기본 파일 정비

  - .gitignore 설정, README.md 초기화, 코드 스타일 설정(ktlint)

5. 기능 개발

- UI 화면 구현
  - 화면 상세 명세서(Figma) 기반 UI 개발: Jetpack Compose 활용
- 상태 관리 및 비즈니스 로직 분리
  - MVVM 패턴 : ViewModel 을 통한 상태 관리
  - UIState / UIEvent 패턴 적용
  - 관심사 분리 : UI(Compose) -> ViewModel -> Repository -> Firebase/API
- 데이터 연동 및 비동기 처리
  - 네트워킹 : Firebase SDK or Retrofit2 + OkHttp Interceptor
  - 비동기 프로그래밍 : coroutines & Flow 를 사용한 비동기 데이터 스트림 처리
  - 의존성 주입 : Hilt를 활용하여 객체 생명주기 및 위존성 관리
- 에러 / 예외 처리 공통화
  - 네트워크 흐름 규격화 : Sealed class 기반의 Result 타입(Success, Error, Loading) 정의
  - 공통 처리 : 네트워크 예외, Firebase 인증 실패, 데이터 파싱 에러 등을 중앙 집중식으로 관리
- 테스트 및 디버깅
  - 단위 개발 검증 : 기능 구현 후 시나리오 기반 개별 테스트
  - Logcal 활용 : 에러 및 예외 상황 점검 및 문제 해결(Troubleshooting) 기록 반복

6. 프로젝트 리뷰 및 리팩토링

- 코드 리뷰 및 공통 스타일 정리

  - 팀원 간 코드 공유를 통해 코드 흐름 및 작성 방식 통일

  - 불필요한 코드 제거 및 네이밍, 들여쓰기, 주석 등 코드 컨벤션 정리

- 중복 코드 제거 및 구조 개선

  - 중복된 로직을 함수/위젯으로 분리하여 재사용성 강화

  - 도메인별 폴더 구조 재정비 및 비즈니스 로직 정리

- 앱 성능 및 사용자 경험 개선

  - 불필요한 상태 변경 제거 및 렌더링 최적화

  - 로딩 인디케이터 처리 및 네트워크 지연 대응 추가

  - UI/UX 요소 정비 (버튼 반응성, 입력 폼 피드백 등)

7. 배포 준비 및 최종 마무리

- 앱 빌드
  - Android용 APK 파일 생성
  - 실제 디바이스 및 브라우저에서 최종 테스트 진행
- 결과물 정리 (스크린샷, 소개 영상 등)
  - 주요 화면 스크린샷 및 소개 영상 제작
  - 앱 사용 흐름 및 기능 시연 자료 구성
- 발표 자료 작성 및 리허설
  - 프로젝트 개요, 개발 흐름, 핵심 기능 위주로 발표 자료 구성
  - 팀원 간 책임 분담 및 발표 리허설 진행
- GitHub 저장소 정리
  - 최종 코드 정리, **README.md** 작성, 프로젝트 정보 명확히 표기

8. 최종 발표

- 팀별 발표 진행 (15~20분 내외)

  - 각 팀별로 프로젝트 진행 결과 발표

  - 앱 시연 중심으로 주요 기능 흐름을 설명

  - 핵심 구현 포인트 및 기술적 도전 과제 공유

- 프로젝트 회고

  - 프로젝트 진행 중 좋았던 점, 아쉬웠던 점, 배운 점 정리

  - 팀 협업에서 느낀 점 및 다음 프로젝트를 위한 개선 방향 도출

  - 기술적 성장 또는 협업 방식에 대한 팀원 개인 회고 포함